VIT BHOPAL UNIVERSITY

SCAI

Book Recommendation System

*Using **Similarity Measured Using L2 Norm***

P R O J E C T R E P O R T

Submitted To
Vityarthi Portal

Submitted By
Lakshya Jain
Regno: 25BAI10994

Academic Year 2025 – 2026 | March 2026

1. Problem Statement

The modern reader faces an overload of choices. With so many books available online, it has become difficult to find one that genuinely fits someone's taste. Traditional search tools rely on keywords and metadata, which cannot reflect the subtle reading patterns that come from real behaviour.

Someone who enjoyed The Alchemist is not simply looking for books marked as philosophical fiction. They want books that other readers with similar interests enjoyed. Genre labels feel too broad and often group together books that only seem similar on the surface.

This project aims to bridge that gap by creating a Book Recommendation System that studies thousands of user ratings and highlights books that share the same audience, without using genres, descriptions, or author details.

This idea is known as collaborative filtering. Instead of asking what a book is about, it looks at who reads it and what else they pick. If readers who love Book A also rate Book B highly, the system sees a strong connection and uses that to make a suggestion.

This approach is widely used by platforms like Netflix, Spotify, and Amazon, and here it is applied to over one million book ratings from the public Book-Crossing dataset.

2. Why This Problem Matters

Recommendation systems sit at the core of every major content platform. In the book world the problem is especially important for several distinct reasons:

- Readers invest significant time in a single book, hours to days, making a poor recommendation far more costly than a wrong song or movie suggestion. A bad film wastes two hours. A bad book recommendation can waste weeks and erodes trust in the platform entirely.
- Unlike movies or music, books are rarely sampled before commitment. There is no 30 second preview. The recommendation must be trustworthy before the reader ever opens the cover.
- The long tail problem is acute. Millions of titles exist, but only a small fraction are ever discovered organically. A recommendation system can surface books that would never appear in a bestseller list but are loved by a specific and well defined reader community.
- Personalisation drives commercial outcomes. Amazon has reported that a substantial portion of its revenue comes from its recommendation engine. In publishing, discoverability is the difference between a book finding its audience and disappearing entirely.
- Different tastes call for different recommendations. Keyword search rewards popular books and punishes niche ones. Collaborative filtering rewards relevance to the specific user's shown preferences, no matter a book's overall popularity.

From an academic view this problem shows core machine learning ideas including collaborative filtering, distance metrics, sparse matrix operations, and careful data preprocessing, all applied to a real dataset of meaningful size.

3. Objective

The specific objectives of this project are clearly defined to span the complete ML pipeline from raw data ingestion through to a deployable interactive application:

- Load, clean, and preprocess the Book-Crossing dataset a real-world collection of over one million book ratings from hundreds of thousands of users.
- Filter the data intelligently to retain only statistically significant users and books, removing noise that would corrupt distance calculations.
- Construct a user-book ratings matrix suitable for distance-based similarity comparison, handling sparsity with a principled fill strategy.
- Implement Euclidean Distance (L2 Norm) as the similarity metric to identify books that have been rated similarly by the same group of users.
- Build an interactive Streamlit web application that lets any user pick a book from a dropdown and receive five personalised recommendations in real time, without requiring any login or user history.

Together these objectives produce a complete, end-to-end system: from a messy CSV file to a live, usable web application demonstrating applied collaborative filtering.

4. Dataset Description

The project uses the publicly available Book dataset,. It remains one of the largest freely available book rating datasets and is widely used in recommender systems research. Two files are consumed, hosted on Google Drive for this project:

File	Key Columns	Description
BX-Books.csv	ISBN, Book-Title	Metadata for ~270,000 books
BX-Book-Ratings.csv	User-ID, ISBN, Book-Rating (0–10)	Over 1,000,000 user-rating interactions

The dataset is intentionally noisy as many users have rated only one or two books, and many books have received just a handful of ratings. Without filtering, these sparse entries would dominate the distance calculations and produce meaningless recommendations.

The Book-Rating column uses a scale of 0 to 10, where 0 represents an implicit interaction (the user interacted with the book but did not provide an explicit rating) and 1–10 represent explicit ratings of increasing quality. This nuance is handled in the preprocessing pipeline.

The preprocessing pipeline described in Section 6 addresses dataset noise by applying activity-based filtering before any modelling occurs which leads to it retaining only users and books that contribute sufficient signal for reliable distance calculations.

5. System Architecture & Workflow

The system follows a clean, linear five-stage pipeline. Each stage produces a well-defined output that feeds directly into the next, making the pipeline easy to reason about, debug, and extend:

- **Load:** Download BX-Books.csv and BX-Book-Ratings.csv from Google Drive via pandas.read_csv() with correct encoding and delimiter settings.
- **Merge:** Join the two datasets on the ISBN key to attach human-readable book titles to every rating row, producing a unified interaction table.
- **Filter:** Remove low-activity users (fewer than 10 ratings) and low-popularity books (fewer than 100 ratings) to reduce noise and improve signal quality.
- **Pivot:** Construct a User × Book matrix where each cell holds the rating given by that user for that book. Missing values are filled with 0.
- **Recommend:** For a selected book, compute Euclidean Distance to every other book in the matrix and return the 5 closest matches as recommendations.

The entire pipeline is wrapped in Streamlit's @st.cache_data decorator so that data loads and processes exactly once per session. Subsequent interactions with the UI are instantaneous because the pivot matrix is held in memory , so no re-download or re-processing occurs between recommendations.

This architecture deliberately keeps all logic in a single Python file, making the project easy to understand, run, and deploy without build systems or containerisation.

6. Methodology

6.1 Data Loading

Both CSV files use semicolons (;) as column separators and latin-1 encoding to handle international characters in book titles. The on_bad_lines='skip' argument gracefully ignores malformed rows without crashing the pipeline.

```
books = pd.read_csv(books_url, sep=";", encoding="latin-1",
on_bad_lines="skip")
ratings = pd.read_csv(ratings_url, sep=";",
encoding="latin-1", on_bad_lines="skip")
```

6.2 Data Cleaning & Filtering

Three cleaning steps are applied to remove noise from the raw data before any modelling occurs:

- Title normalisation: All titles are lowercased and stripped of whitespace, preventing case-sensitivity duplicates (e.g., 'Harry Potter' and 'harry potter' treated as the same book).
- User filtering: Users who rated fewer than 10 books are dropped — they contribute too little signal to meaningfully influence distance calculations.
- Book filtering: Books rated by fewer than 100 distinct users are removed; recommendations for obscure books would rest on too few data points to be reliable.

```
books["Book-Title"] = books["Book-
Title"].str.lower().str.strip()

user_counts = data["User-ID"].value_counts()
active_users = user_counts[user_counts >= 10].index
data = data[data["User-ID"].isin(active_users)]

book_counts = data["Book-Title"].value_counts()
popular_books = book_counts[book_counts >= 100].index
data = data[data["Book-Title"].isin(popular_books)]
```

6.3 Pivot Matrix Construction

The filtered data is reshaped into a pivot table — a 2D matrix where rows represent users, columns represent books, and each cell holds the rating given. Missing cells (where a user has not rated a book) are filled with 0.

```
pivot = data.pivot_table(index="User-ID", columns="Book-
Title", values="Book-Rating")
pivot = pivot.fillna(0)
```

The resulting matrix is inherently sparse. Using 0 as the fill value treats unrated books neutrally — it does not infer a negative or positive signal where none exists.

6.4 Euclidean Distance (L2 Norm)

Each book is represented as a high-dimensional vector — one dimension per user, each value being that user's rating. To measure similarity between two books, the system computes the Euclidean Distance (L2 Norm) between their vectors:

```
distance(A, B)  =  sqrt [ sum ( a_i - b_i )^2 ]
```

Where a and b are the rating vectors of two books and i iterates over every user. A small distance means both books received very similar ratings from the same group of users — indicating they appeal to the same audience.

```
distance = np.sqrt(np.sum((selected_book_ratings -
other_book_ratings) ** 2))
```

Why Euclidean Distance over cosine similarity? Cosine similarity ignores vector magnitude: a user giving all 2s and another giving all 10s would appear identical to cosine similarity. Euclidean distance captures actual differences in rating magnitude, making it more meaningful in a star-rating context.

6.5 Top-5 Recommendation

After computing distances to all books in the pivot matrix, results are sorted in ascending order of distance. The query book itself is skipped, and the top 5 closest books are returned.

```
sorted_books = sorted(distances, key=lambda b:
distances[b])
return sorted_books[:5]
```

The returned titles are capitalised using Python's .title() method before display, converting the normalised lowercase titles back into a readable form for the user interface. The entire recommendation step runs in milliseconds.

7. Key Design Decisions

- **Threshold of ratings (users & books) -** Lowering the user threshold introduces noisy users with too little data, while increasing it shrinks the dataset too much. A minimum of 10 ensures meaningful user profiles. Similarly, books with very few ratings lack reliable signals and appear artificially similar to others, so a 100-rating cutoff ensures better comparison quality.

- **Fill NA with 0 (not mean) -** Mean Imputation falsely assumes users rated unread books. Using 0 correctly represents no interaction and avoids corrupting similarity calculations.

- **Euclidean Distance over Cosine Similarity -** Cosine ignores rating magnitude, treating different rating scales as identical. Euclidean distance captures actual differences in preferences, making it more suitable for rating-based systems.

- **Drop duplicates before pivot -** Duplicate user-book ratings cause pivot errors. Removing them ensures a clean and consistent matrix.

8. Challenges Faced

- **Data Quality & Encoding -** The dataset uses non-standard settings (latin-1, semicolons), requiring custom parameters in `pd.read_csv()` to avoid errors or corrupted data.

- **Sparsity of Rating Matrix -** Most entries are zero, meaning users haven't rated most books. This can distort similarity, as shared zeros may falsely imply similarity.

- **Scalability of Distance Loop -** Computing distance for every book is O(n) and can slow down with large datasets. Precomputing similarities would improve performance.

- **Case Sensitivity & Duplicates -** Inconsistent capitalization creates duplicate entries for the same book. Lowercasing ensures a unified and accurate dataset.

- **Cold Start Problem -** New users or books lack sufficient data for recommendations, a known limitation of collaborative filtering.

9. Results & Output

The system successfully loads and processes the Book-Crossing dataset, constructing a working recommendation engine that operates in real time. The results are semantically coherent — books recommended together genuinely share a readership demographic, even though the system has no knowledge of genre, author, or content.

A representative output for the query 'The Secret Garden' is shown below. All five recommended books belong to the same late-19th / early-20th century children's classic genre:

Rank	Recommended Book	Audience Overlap (Inferred)
1	A Little Princess	Shared classic-literature readership
2	Little Women	Overlapping young-adult audience
3	Anne of Green Gables	Similar era and protagonist profile
4	The Wind in the Willows	Users who enjoy nature-themed classics
5	Black Beauty	Overlapping historical fiction readers

This output validates that the distance metric is capturing genuine audience overlap rather than producing random matches. None of these recommendations rely on genre tags or descriptions — they emerge entirely from patterns in how real users distributed their ratings across books.

The Streamlit interface loads in under a minute on first run (including dataset download and preprocessing) and returns recommendations instantaneously on subsequent queries, thanks to the caching layer described in Section 5.

Sample results were consistent across multiple test queries — books with well-defined readership communities produced tight, coherent recommendation clusters, while genre-blending books produced more eclectic recommendations that nonetheless reflected real patterns in the data.

10. Libraries & Tools Used

All libraries used in this project are open-source, actively maintained, and pip-installable. The entire environment can be set up with a single command: pip install streamlit pandas numpy.

Library / Tool	Version	Purpose
Python	3.10+	Core programming language for all logic
pandas	2.x	Data loading, merging, filtering, and pivot table construction
NumPy	1.24+	Vectorised Euclidean distance computation
Streamlit	1.x	Interactive web application framework and UI layer
Google Drive	—	Remote hosting for the CSV dataset files

The application is launched via the command:

```
python -m streamlit run book_recommender.py
```

No additional configuration or environment variables are required. The dataset is downloaded automatically from Google Drive on first run and cached in memory for the duration of the session by Streamlit's caching decorator.

NumPy is the critical performance dependency — without vectorised array operations, computing Euclidean distance across thousands of book pairs per query would be prohibitively slow in pure Python. NumPy reduces each distance calculation to a single expression operating on array slices.

11. What I Learned

Data Quality & Encoding - Non-standard format (latin-1, semicolons) needs custom `read_csv()` settings to avoid errors.

Sparsity of Rating Matrix - Many zeros distort similarity, as shared missing ratings can appear meaningful.

Scalability of Distance Loop - O(n) computation slows at scale; precomputing similarities improves speed.

Case Sensitivity & Duplicates - Inconsistent titles create duplicates; lowercasing fixes this.

Cold Start Problem - New users/books lack data, limiting recommendations.

12. Future Scope

Matrix Factorisation (SVD/ALS) - Handles sparsity by learning latent patterns in the user-book matrix.

Content-Based + Hybrid Filtering - Uses genres, authors, and TF-IDF to solve cold start and improve recommendations.

Precomputed Similarity Matrix - Eliminates per-request computation, making recommendations faster.

User-Based Collaborative Filtering - Recommends books liked by similar users for better personalisation.

Cloud Deployment - Hosting on Streamlit Cloud/AWS/GCP enables public access without local setup.

Cover Image Integration - Fetching book covers via APIs improves UI and user engagement.

Feedback Loop - User ratings refine the model over time, improving recommendation quality.

13. Conclusion

This project shows the complete development of a collaborative filtering based book recommendation system, from raw data to a working application. Starting with a dataset of over a million user interactions, the system processes and cleans the data, then uses a simple and understandable distance method to generate recommendations. All of this is done without using extra information like genres, descriptions, or author details.

Along the way, several practical decisions were made. For example, choosing Euclidean distance instead of cosine similarity, filling missing values with zero instead of averages, setting minimum thresholds for active users and popular books, and removing duplicate entries before building the matrix. Each of these choices was made to keep the system simple, consistent, and effective, and being able to explain them is an important part of understanding applied machine learning.

The project also highlights real challenges that come with recommendation systems. These include messy data, encoding issues, sparse matrices, performance limitations of comparing every book, handling text inconsistencies, and the cold start problem. These are not flaws in the project but natural difficulties that appear in real world systems, and recognizing them is an important learning outcome.

In the end, the result is a working application that users can interact with. A user selects a book, and the system returns five similar recommendations almost instantly, based on patterns in how readers have rated different books.

Overall, the project shows that even a simple collaborative filtering approach can produce meaningful recommendations using only user behavior. At the same time, it also makes it clear where improvements are needed for scaling this into a more advanced production system.